

```
In [1]: %run Latex_macros.ipynb
```

```

 $\mathbf{x}$  \newcommand{\tx}{\tilde{\mathbf{x}}} \newcommand{\y}
{\mathbf{y}} \newcommand{\b}{\mathbf{b}} \newcommand{\c}{\mathbf{c}}
\newcommand{\e}{\mathbf{e}} \newcommand{\z}{\mathbf{z}} \newcommand{\h}
{\mathbf{h}} \newcommand{\u}{\mathbf{u}} \newcommand{\v}{\mathbf{v}}
\newcommand{\w}{\mathbf{w}} \newcommand{\V}{\mathbf{V}} \newcommand{\W}
{\mathbf{W}} \newcommand{\X}{\mathbf{X}} \newcommand{\KL}{\mathbf{KL}}
\newcommand{\E}{{\mathbb{E}}} \newcommand{\Reals}{{\mathbb{R}}}
\newcommand{\ip}{{\mathbf{(i)}}} % % Test set \newcommand{\xt}{\underline{\mathbf{x}}}
\newcommand{\yt}{\underline{\mathbf{y}}} \newcommand{\Xt}{\underline{\mathbf{X}}}
\newcommand{\perfm}{\mathcal{P}} % % \l indexes a layer; we can change the actual
letter \newcommand{\ll}{l} \newcommand{\llp}{{(\ll)}} % \newcommand{\Thetam}
{\Theta_{-0}} % CNN \newcommand{\kernel}{\mathbf{k}} \newcommand{\dim}{d}
\newcommand{\idxspatial}{{\text{idx}}} \newcommand{\summaxact}{{\text{max}}}
\newcommand{\idxb}{\mathbf{i}} % % % RNN % \tt indexes a time step
\newcommand{\tt}{t} \newcommand{\tp}{{(\tt)}} % % % LSTM \newcommand{\g}
{\mathbf{g}} \newcommand{\remember}{\mathbf{remember}} \newcommand{\save}
{\mathbf{save}} \newcommand{\focus}{\mathbf{focus}} % % % NLP
\newcommand{\Vocab}{\mathbf{V}} \newcommand{\v}{\mathbf{v}}
\newcommand{\offset}{o} \newcommand{\o}{o} \newcommand{\Emb}{\mathbf{E}} % %
\newcommand{\loss}{\mathcal{L}} \newcommand{\cost}{\mathcal{L}} % %
\newcommand{\pdata}{p_{\text{data}}} \newcommand{\pmodel}{p_{\text{model}}} % %
SVM \newcommand{\margin}{{\mathbb{m}}} \newcommand{\lmk}{\boldsymbol{\ell}} %
% % LLM Reasoning \newcommand{\rat}{\mathbf{r}} \newcommand{\model}
{\mathcal{M}} \newcommand{\bthink}{\text{}} \newcommand{\ethink}{\text{}} % % %
Functions with arguments \def\xsy#1#2{\#1^{\#2}} \def\rand#1{\tilde{\#1}}
\def\randx{\rand{\mathbf{x}}} \def\randy{\rand{\mathbf{y}}} \def\trans#1{\dot{\#1}}

```

```

\def\transx{\trans{\x}}\def\transy{\trans{\y}}% \def\argmax#1{\underset{#1}
{\operatorname{argmax}}} \def\argmin#1{\underset{#1}{\operatorname{argmin}}}
\def\max#1{\underset{#1}{\operatorname{max}}} \def\min#1{\underset{#1}
{\operatorname{min}}} % \def\pr#1{\mathcal{p}(\#1)} \def\prc#1#2{\mathcal{p}(\#1 \; |
\; \#2)} \def\cnt#1{\mathcal{count}_{\#1}} \def\node#1{\mathbb{\#1}} %
\def\Exp#1{\underset{#1}{\operatorname{Exp}}} \def\ExpOf#1{\mathcal{O}\left(\#1\right)} %
Expectation operator \def\Exp#1{\underset{#1}{\operatorname{Exp}}} % %
By now, it has become common to be able to generate images via textual descriptions.
VAE \def\prs#1#2{\mathcal{p}_{\#2}(\#1)} \def\qr#1{\mathcal{q}(\#1)}
\def\qrs#1#2{\mathcal{q}_{\#2}(\#1)} % % Reinforcement learning
To illustrate (this is a very early implementation, current implementations are much
better)
\newcommand{\Actions}{{\mathcal{A}}} \newcommand{\actseq}[1]{
\newcommand{\act}[1]{a} \newcommand{\States}{{\mathcal{S}}}
\newcommand{\stateseq}[1]{S} \newcommand{\state}[1]{s} \newcommand{\Rewards}
{{\mathcal{R}}} \newcommand{\rewseq}[1]{R} \newcommand{\rew}[1]{r}
\newcommand{\transp}[1]{P} \newcommand{\statevalfun}[1]{v} \newcommand{\actvalfun}
[q] \newcommand{\disc
\right\rceil} \newcomr

```

Mixing Text and Images

Text input: "An illustration of a baby daikon radish in a tutu walking a dog"

Image output:



Let's imagine how we might achieve this.

For text (sequence of language tokens)

- Transformer LLM
 - trained on Language Modeling task ("predict the next" token)

But how do achieve the ability to process Images ?

- as output
 - Text to Image: "An illustration of a baby daikon radish in a tutu walking a dog"
- as input
 - Image to Text: "Find my wallet in the photo of my living room"

A desirable goal would be

- enable a Transformer to handle both Text and Image

But there are some obvious impediments to using a Transformer for non-Text

- e.g., Image, Audio

To start

- images are large
 - a low resolution image these days is at least 1 million pixels
 - Context length of LLM's is a limitation

Additionally

- the "shape" of images is not the same as Text
 - images have a non-feature dimension (Spatial)
 - text has a non-feature dimension (Temporal)

Perhaps the most intimidating challenge

- Transformers operate on categorical variables (tokens)
- Image pixels are *continuous* variables

Preview of a solution

The solution will be

- reduce an Image to a sequence of "Image tokens"
 - a sequence of categorical values (Image tokens)
 - that represent the Image

Once we have the idea of Image Tokens

- we can mix Image and Text tokens in a Transformer

That solves the "syntax" problem of mixing Text and Images.

But how do we train the Transformer to create Images ?

Answer: adapt the Language Modeling task !

Remember: an Image is now represented as a sequence of Image tokens.

So the Language Modeling task for Images becomes

- Predict the next Image Token, conditional on a prefix of the sequence of Image tokens for an Image

We can train the Transformer on the "predict the next" task

- where the predictions are Image tokens, rather than Text tokens

What about solving the Text to Image task ?

Let's create a sequence with two sub-sequences

- the initial sub-sequence are Text tokens that describe an Image
- the final sub-sequence are the Images tokens of an Image matching the description

Example: <text token> <text token> ... <text token> [SEP] <image token> <image token> ...

What happens if we use this example to train a generalized (mixed Text and Image) "predict the next" task ?

Hopefully, training will cause the model, when given a description

`<text token> <text token> ... <text token> [SEP]`

to produce a continuation (sequence of next tokens) that are only of type Image token

`<image token> <image token> ...`

where the continuation are Image tokens matching the Text description !

Here is what a training example that mixes text and an image looks like:

Training example: Representing mixed Text + Image as a sequence

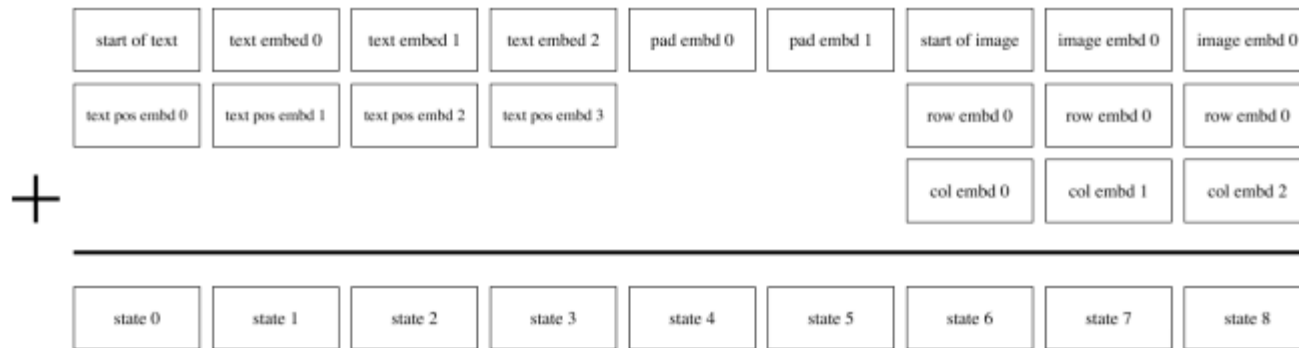


Figure 10. Illustration of the embedding scheme for a hypothetical version of our transformer with a maximum text length of 6 tokens. Each box denotes a vector of size $d_{\text{model}} = 3968$. In this illustration, the caption has a length of 4 tokens, so 2 padding tokens are used (as described in Section 2.2). Each image vocabulary embedding is summed with a row and column embedding.

Notice the positional embeddings

- position in text
- row/column position for image

Image tokenization

Thus, the main impediment to mixing Text and Image is representing an

- Image (spatial arrangement of continuous pixel values)
- as a sequence of Image tokens

The Discrete Variational Autoencoder (dVAE) is a "tokenizer" for Images.

It maps

- Input: 3D RGB pixels
- Output: sequence of vectors

The key insight

- each vector in the sequence
- is chosen from a **pre-defined finite set of vectors** (the *codebook*)

The codebook is the "Vocabulary" of Image tokens.

Thus an Image token can be represented as a Categorical value

- position within the codebook

How is the codebook (Vocabulary) of pre-defined vectors created ?

As a by-product of Training !

- the vectors are learned parameters of a model

Controlling the length of Image token sequences

Creating one Image token per pixel

- would result in the need for the Transformer to have a very large Context window

In practice

- the input Image
- is reduced to "patches"
 - small pixel grids (e.g., 32×32)
- and we create Image tokens for patches

In [2]: `print("Done")`

Done

